Stat 20 Summer 2026 — Review Session 2

Sampling Distributions, Hypothesis Tests, Causation, and Prediction

Curtis McDonald

Table of contents

How to use these notes. Each section states the *idea* first, then shows the *R code* you'd actually run. All examples use the `penguins` data from `stat20data`, so you can copy any chunk into RStudio and run it. Load the libraries once at the top of your session:

```
library(tidyverse)
library(stat20data)
library(broom)
library(infer)
data(penguins)
```

1 The Normal Approximation

1.1 iid random variables, sums, and averages

$X_1, \ldots, X_n$ are **iid** (independent and identically distributed) if they are independent draws from the *same* distribution — the box model: draw n tickets **with replacement** from one box. Let each X_i have mean μ and SD σ. Define the **sum** $S_n = X_1 + \cdots + X_n$ and the **average** $\bar{X} = S_n/n$. Both are themselves random variables, and:

	Mean	Variance	SD
Sum S_n	$n\mu$	$n\sigma^2$	$\sqrt{n}\,\sigma$
Average $\bar{X}$	μ	σ^2/n	$\sigma/\sqrt{n}$

This is the **square root law**: the spread of a sum grows like $\sqrt{n}$ (slower than the mean, which grows like n), and the spread of an average *shrinks* like $1/\sqrt{n}$. Note that $E(\bar{X}) = \mu$: the sample mean is centered on the population mean, and its variance $\sigma^2/n \to 0$ as n grows.

1.2 The Central Limit Theorem (CLT)

CLT (course version): the sum (or average) of n iid random variables, after standardizing, follows an approximately **normal** distribution as n grows large — *regardless* of the distribution of the tickets in the box.

Two important caveats to keep your statement precise:

- It is *not* the CLT that gives $E(S_n) = n\mu$ and $\text{Var}(S_n) = n\sigma^2$ — those follow from linearity of expectation and additivity of variance for independent variables. The CLT is a statement about the **shape**: $S_n \approx N(n\mu,\, n\sigma^2)$ and $\bar{X} \approx N(\mu,\, \sigma^2/n)$.
- The approximation is good for "small deviations" from the expected value (on the order of $\sqrt{n}\,\sigma$, i.e. a few SEs). Don't use it for questions about wildly extreme values far outside that range.

Since most statistics we care about ($\bar{x}$, $\hat{p}$, differences of means or proportions, regression coefficients) are sums or averages in disguise, their sampling distributions are approximately normal at large n. That's what powers the normal-approximation confidence interval below.

1.3 The Normal distribution in R

If $X \sim N(\mu, \sigma^2)$, then R gives the usual trio (parametrized by the **SD**, not the variance):

- `dnorm(x, mean, sd)` — density $f(x)$
- `pnorm(q, mean, sd)` — CDF $P(X \le q)$
- `rnorm(n, mean, sd)` — random draws

The **empirical rule** (68 / 95 / 99.7% within 1 / 2 / 3 SDs) is just areas under the normal curve:

```
pnorm(1) - pnorm(-1)      # ~ 0.68
```

```
[1] 0.6826895
```

```
pnorm(2) - pnorm(-2)      # ~ 0.95
```

```
[1] 0.9544997
```

```
pnorm(1.96) - pnorm(-1.96) # exactly 95% -- where the 1.96 comes from
```

```
[1] 0.9500042
```

Exam move

"Roulette / dice / drunkard's walk" problems: (1) find the mean and SD of *one* draw; (2) scale by n (sum) or $1/n$ (average) using the table above; (3) invoke the CLT to treat the sum/average as normal; (4) convert to standard units $z = (\text{value} - \text{mean})/\text{SD}$ and use `pnorm()`.

2 From Samples to Populations

2.1 The four key terms

Term	Meaning	Symbol conventions
Population	the full set of units you want to learn about	size N; parameters get Greek letters (μ, σ, p)
Sample	the subset of units you observed (your data)	size n
Statistic	numerical summary of the *sample*	Latin letters: $\bar{x}$, s, $\hat{p}$, b_1
Population parameter	the analogous summary of the *population*	μ, σ, p, β_1 — usually unknown

2.2 The sampling distribution

Sampling distribution The distribution of a *statistic* under repeated sampling — all the values the statistic *could* have taken across the samples you *could* have drawn, with their probabilities.

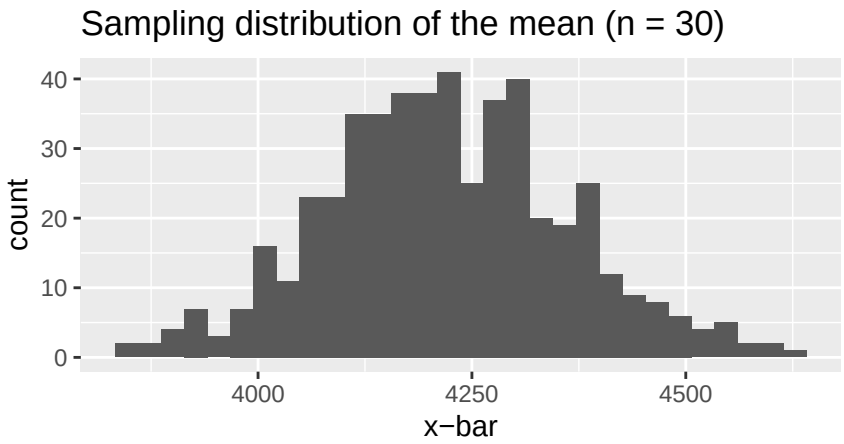
This is neither the population distribution nor the empirical (data) distribution. It is usually hypothetical (you only see one sample), but it is the object that tells you how far your statistic can stray from the truth. For the mean of a large SRS, the three-panel picture is:

	Population	Empirical (your sample)	Sampling dist. of $\bar{x}$
Shape	whatever it is	resembles population	bell-shaped (CLT)
Center	μ	$\bar{x} \approx \mu$	μ
Spread	σ	$s \approx \sigma$	$SE = \sigma/\sqrt{n}$ (small!)

Simulating it (a thought-experiment where we pretend `penguins` is the population): draw many samples and compute many $\bar{x}$s.

```
set.seed(20)
xbars <- penguins |>
  rep_slice_sample(n = 30, replace = FALSE, reps = 500) |>
  group_by(replicate) |>
  summarize(xbar = mean(body_mass_g, na.rm = TRUE))

ggplot(xbars, aes(x = xbar)) +
  geom_histogram(bins = 30) +
  labs(title = "Sampling distribution of the mean (n = 30)", x = "x-bar")
```



(`slice_sample()` draws one sample; `rep_slice_sample()` from `infer` draws `reps` samples and tags them with a `replicate` column; `weight_by =` makes some rows more likely — how we simulate selection bias.)

2.3 Sources of error: bias and variance

Think of throwing darts. **Bias** = systematically off-center, bias $= E(\bar{X}) - \mu^*$. **Variance** = erratic scatter, $\text{Var}(\bar{X})$. A representative sampling method has bias 0, so $E(\bar{X}) = \mu^*$.

Four sources of bias:

1. **Selection bias** — units are not equally likely to be sampled (e.g. convenience samples, self-selected volunteers). Fixed by a *simple random sample* (SRS): every unit equally likely.
2. **Measurement bias** — the measuring process systematically misses in one direction (leading survey questions, drifting instruments).

3. **Non-response bias** — selected units fail to respond, and responders differ from non-responders.
4. **Intentional design bias** — a statistician *chooses* a biased estimator on purpose because lower variance more than pays for the small bias (the bias–variance trade-off, which returns in prediction).

Two sources of variance:

1. **Sampling variability** — a different random sample gives a different statistic.
2. **Measurement variability** — measuring the same unit twice gives different values.

Exam move

Given a data-collection story, name the error source *and* its direction. "The professor polled the front row" → convenience sample → selection bias (eager first-years over-represented → estimate too low). "First-years lie about their year" → measurement bias (estimate too high). Bias survives a bigger sample; variance shrinks with n.

3 Standard Error and Confidence Intervals

3.1 Standard Error (SE)

Standard Error The standard deviation of the sampling distribution of a statistic. (The special name keeps it distinct from the SD of the data.)

For a sample mean from an SRS, $SE(\bar{x}) = \sigma/\sqrt{n}$; since σ is unknown, plug in the sample SD s:

$$SE(\bar{x}) \approx \frac{s}{\sqrt{n}}$$

For a **0–1 (Bernoulli) population** with proportion p of 1s, one draw has $\sigma = \sqrt{p(1-p)}$, so

$$SE(\hat{p}) \approx \frac{\sqrt{\hat{p}(1-\hat{p})}}{\sqrt{n}}.$$

Two facts the SE table drives home: accuracy improves like $1/\sqrt{n}$ (a sample of 400 is *twice* as accurate as a sample of 100, not 4 times), and the **population size N doesn't matter** as long as n is small relative to N.

```r
penguins |>
  drop_na(body_mass_g) |>
  summarize(xbar = mean(body_mass_g),
            s    = sd(body_mass_g),
            n    = n(),
            SE   = s / sqrt(n))
```

```
# A tibble: 1 x 4
   xbar     s     n    SE
  <dbl> <dbl> <int> <dbl>
1 4207.  805.   333  44.1
```

3.2 What a confidence interval means

Confidence interval (CI) An interval of values, built from the sample, designed to capture the population parameter.

Correct interpretation of "95%": *under repeated sampling*, 95% of the intervals constructed this way will contain the population parameter. Once your one interval is computed, it either contains the parameter or it doesn't — there is nothing random left. So say "I am 95% *confident* μ is in [a, b]", **NOT** "there is a 95% *probability* that μ is in [a, b]". The randomness was in drawing the sample, not in μ.

3.3 TWo ways to build a 95% CI

3.3.1 1. Normal approximation CI

The CLT says the sampling distribution is roughly normal, so

$$[\,\bar{x} - 1.96\,SE,\;\; \bar{x} + 1.96\,SE\,] \qquad \text{with } SE = \frac{s}{\sqrt{n}} \;\; \left(\text{or } \frac{\sqrt{\hat{p}(1-\hat{p})}}{\sqrt{n}} \text{ for a proportion}\right)$$

```r
penguins |>
  drop_na(body_mass_g) |>
  summarize(xbar = mean(body_mass_g),
            SE   = sd(body_mass_g) / sqrt(n()),
            lower = xbar - 1.96 * SE,
            upper = xbar + 1.96 * SE)
```

```
# A tibble: 1 x 4
   xbar    SE lower upper
  <dbl> <dbl> <dbl> <dbl>
1 4207.  44.1 4121. 4294.
```

3.3.2 2. Bootstrap CI (percentile method)

When the statistic isn't a nice sum, or n is modest, approximate the sampling distribution by **resampling the sample**:

1. Treat the sample as the *bootstrap population*.
2. Draw a new sample of the **same size n, WITH replacement**.
3. Compute the statistic on the resample.
4. Repeat many times (use 500 in this class) $\rightarrow$ the *bootstrap distribution*.
5. 95% CI = the 0.025 and 0.975 quantiles of that distribution.

```r
set.seed(20)
boot <- penguins |>
  specify(response = body_mass_g) |>
  generate(reps = 500, type = "bootstrap") |>
  calculate(stat = "mean")

get_ci(boot, level = 0.95)
```

```
# A tibble: 1 x 2
  lower_ci upper_ci
     <dbl>    <dbl>
1    4121.    4300.
```

The `infer` pipeline is always the same four verbs: `specify()` (which variable(s)) `|> generate()` (make replicate data sets) `|> calculate()` (one statistic per replicate) `|> get_ci()` / `visualize()`. For a proportion, name the level you're counting: `specify(response = sex, success = "female")` and `calculate(stat = "prop")`. For a regression coefficient, use `specify(y ~ x1 + x2) |> generate(...) |> fit()` and then `filter(term == "x1")` before `get_ci()`.

What is a valid bootstrap sample/distribution? Check for:

- drawn from **the original data only** — repeated rows are expected, no brand-new values, no shuffled/recombined columns;
- **same size as the original** data frame (n resampled rows per replicate);
- sampled **with replacement** (some rows appear twice+, others not at all).

4 Hypothesis Testing

4.1 The anatomy of a test

Every hypothesis test, whatever the context, has the same four parts:

1. **Null hypothesis** H_0 — a specific chance model for how the data could be generated ("nothing interesting is going on"). State it in terms of *parameters*: $p_1 - p_2 = 0$, not $\hat{p}_1 - \hat{p}_2 = 0$.
2. **Alternative hypothesis** H_A — some other mechanism generated the data ($p_1 - p_2 \neq 0$, or > 0, or < 0).
3. **Test statistic** — one number computed from the data that bears on H_0 (a mean, difference in props, χ^2, ...). Under H_0 it has a sampling distribution called the **null distribution**.
4. **p-value** — the probability, *assuming* H_0 *is true*, of a test statistic as extreme or more extreme than the one observed.

Small p-value → the data are inconsistent with H_0 → **reject** H_0. Large p-value → **fail to reject** H_0 (never "accept" / "the null is true" — think "not guilty", not "innocent").

What a p-value is NOT:

1. NOT the probability that H_0 is true — it's computed *assuming* H_0.
2. A high p-value does NOT show H_0 is true — absence of evidence, not evidence of absence.
3. NOT "the probability of the data" — it's the probability of the *test statistic* being this extreme, *given* the null.
4. It is a probability, so if you compute -6 or 3.2, go find your bug.

4.2 Decision errors, α, and power

	Reject H_0	Fail to reject H_0
H_0 **true**	**Type I error** (false positive)	correct
H_A **true**	correct	**Type II error** (false negative)

- **Significance level** α: the p-value threshold, chosen *before* the test (convention 0.05). It IS the Type I error rate: a level-0.05 test rejects a true null 5% of the time — at *any* sample size. More data does not lower α.
- **Power** $= 1 - \beta$: the probability of rejecting H_0 when the alternative is true ($\beta = \text{P(Type II error)}$). Power grows with sample size and with effect size. A high p-value from a tiny study may just be a low-powered study.
- The trade-off: pushing one error down pushes the other up. Costly false positives → lower α (0.01); costly false negatives → higher α (0.10).

One-sided vs two-sided. $H_A :\neq$ splits the rejection region into both tails (2.5% each at $\alpha = .05$); $H_A :>$ or $<$ puts all 5% in one tail. The choice must be made **before seeing the data** — flipping to the "matching" one-sided test afterward doubles your true Type I rate to 10% (p-hacking). Default to two-sided; for a two-sided p-value, compute one tail and double it. (The χ^2 test is the standard exception where one-sided-right is correct: only large distances signal inconsistency with the null.)

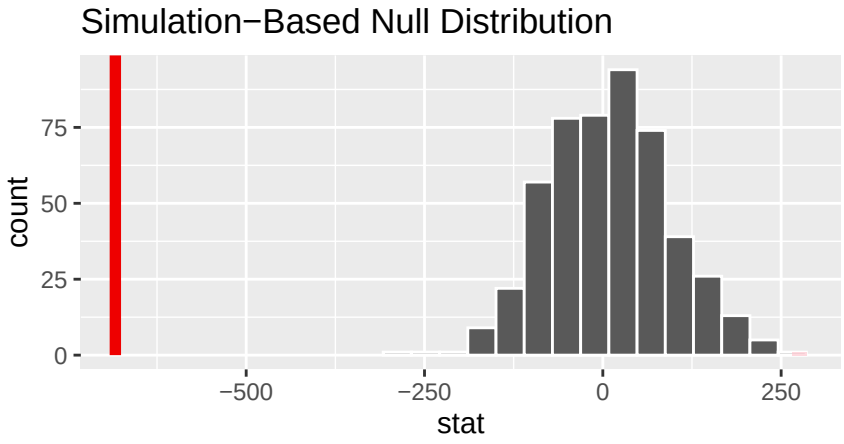
4.3 Test 1: Permutation test (two groups)

Null: the response is **independent** of the group labels. Simulate that world by **shuffling** which rows are in which group, recomputing the statistic each time. With `infer`, add `hypothesize(null = "independence")` and `generate(type = "permute")`:

```
# Is mean body mass different for female vs male penguins?
obs_stat <- penguins_c |>
  specify(response = body_mass_g, explanatory = sex) |>
  calculate(stat = "diff in means", order = c("female", "male"))

set.seed(20)
null <- penguins_c |>
  specify(response = body_mass_g, explanatory = sex) |>
  hypothesize(null = "independence") |>
  generate(reps = 500, type = "permute") |>
  calculate(stat = "diff in means", order = c("female", "male"))

null |>
  visualize() +
  shade_p_value(obs_stat, direction = "both")
```



```r
get_p_value(null, obs_stat, direction = "both")
```

```
# A tibble: 1 x 1
  p_value
    <dbl>
1       0
```

(For two proportions — treatment vs control — the code is identical with `success = "..."` in `specify()` and `stat = "diff in props"`.)

4.4 Test 2: Point null — sampling from a specified distribution

Null: the data came from a **fully specified** distribution, e.g. "the coin is fair, Bernoulli(0.5)" or "first digits follow Benford's Law". Simulate by **drawing tickets from a box** built to match the null: `hypothesize(null = "point", ...)` + `generate(type = "draw")`.

```r
# Is the proportion of female penguins 0.5?
obs_prop <- penguins_c |>
  specify(response = sex, success = "female") |>
  calculate(stat = "prop")

set.seed(20)
null_prop <- penguins_c |>
  specify(response = sex, success = "female") |>
  hypothesize(null = "point", p = 0.5) |>
  generate(reps = 500, type = "draw") |>
  calculate(stat = "prop")

get_p_value(null_prop, obs_prop, direction = "both")
```

```
# A tibble: 1 x 1
  p_value
    <dbl>
1   0.904
```

4.5 The chi-squared statistic (many proportions)

When the null specifies a whole categorical distribution (k levels with probabilities $p_1, \ldots, p_k$), measure the **distance** between observed and expected counts with

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}, \qquad E_i = n \times p_i.$$

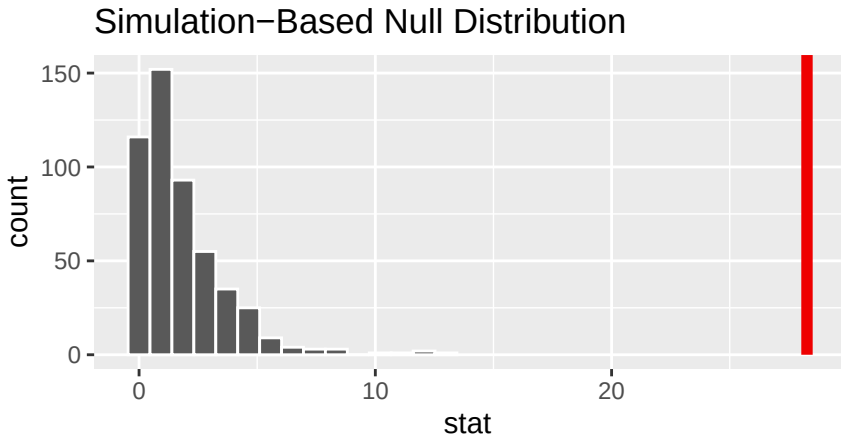
Squaring treats + and - misses alike; dividing by E_i makes each term relative to its expected bar height. $\chi^2 = 0$ means a perfect match; big values mean big distance — but "big" only has meaning relative to the null distribution of χ^2, which you build by drawing from the null.

```
# Toy null: the three species are equally common (p = 1/3 each)
p_null <- c("Adelie" = 1/3, "Chinstrap" = 1/3, "Gentoo" = 1/3)

obs_chi <- penguins_c |>
  specify(response = species) |>
  hypothesize(null = "point", p = p_null) |>
  calculate(stat = "Chisq")

set.seed(20)
null_chi <- penguins_c |>
  specify(response = species) |>
  hypothesize(null = "point", p = p_null) |>
  generate(reps = 500, type = "draw") |>
  calculate(stat = "Chisq")

null_chi |>
  visualize() +
  shade_p_value(obs_chi, direction = "right")   # one-sided: right tail only
```



Exam move

Given any study, answer four questions: (1) what is H_0? (2) what is the observed test statistic? (3) how was the null distribution approximated — *permute* (shuffle labels; independence null) or *draw* (tickets from a box; point null) or normal theory? (4) what is the p-value, and what does it (not) tell us? Also match code to null: `type = "permute"` ↔ `null = "independence"`; `type = "draw"` ↔ `null = "point"`; `type = "bootstrap"` ↔ no `hypothesize()` at all (that's a CI, not a test).

5 Causation and Experiments

5.1 Counterfactuals and the ATE

"A causes B" (conditional counterfactual definition): in a world where A didn't happen, B no longer happens. The fundamental problem: for one individual you only ever observe *one* of the two **potential outcomes** (treated or not), never both. So individual-level causal claims are out of reach with data alone — but group-level claims are estimable:

Average Treatment Effect (ATE) A population parameter capturing the effect of a treatment averaged over a group. Estimated by a difference in sample means ($\bar{x}_T - \bar{x}_C$) or proportions ($\hat{p}_T - \hat{p}_C$), comparing treatment and control groups.

The catch: $\hat{p}_T - \hat{p}_C$ only estimates a *causal* effect if the control group is a good stand-in for the treatment group's counterfactual. That is what experimental design buys you.

5.2 Principles of a randomized controlled trial (RCT)

- **Control**: a second condition to contrast with treatment; controlling (verb) means removing other differences between groups.
- **Random assignment**: each subject is equally likely to land in treatment or control. This balances *every* other characteristic between groups **on average** — measured or not — cutting off "maybe it was really X" objections.
- **Replication**: many subjects per group (and repeated studies). Randomization only balances on average; larger n makes bad-luck imbalance very unlikely.
- **Blinding**: subjects don't know their group (single-blind, via a **placebo**); ideally neither do the researchers interacting with them (double-blind). Controls for the placebo effect and differential care.

Random *assignment* vs random *selection* — two different kinds of randomness that license two different conclusions:

	Random selection (sampling)	Random assignment
What it buys	**generalization** to the population	**causal** interpretation of the group difference
Fixes	selection bias	confounding

A study can have either, both, or neither; classify what claim each supports.

5.3 Covariates, balance, and Love plots

Covariate A variable measured (or determined) *before* treatment was administered — so it cannot have been affected by the treatment.

Covariate balance A covariate is balanced if its distribution in the treated group is similar to its distribution in the control group. Randomization produces balance *on average*.

The standard summary for one numerical covariate x is the **standardized mean difference**:

$$SMD(x) = \frac{\bar{x}_{treatment} - \bar{x}_{control}}{\sqrt{\frac{1}{2}\hat{\sigma}^2_{treatment} + \frac{1}{2}\hat{\sigma}^2_{control}}}$$

— a difference in means divided by a pooled SD so covariates on different scales are comparable. A **Love plot** displays the SMD of every covariate on a common axis (one dot per covariate, vertical line at 0), so you can see at a glance which covariates are most imbalanced and in which direction. Small non-zero SMDs are expected under randomization; to judge whether an imbalance is more than chance, run a permutation test *using the covariate as the response*.

```
# Simulate an RCT on penguins and check balance of a covariate
set.seed(20)
expt <- penguins_c |>
  mutate(group = sample(c("treatment", "control"), n(), replace = TRUE))

# SMD for the covariate body_mass_g
expt |>
  group_by(group) |>
  summarize(m = mean(body_mass_g), v = var(body_mass_g)) |>
  summarize(SMD = (m[group == "treatment"] - m[group == "control"]) /
                  sqrt(0.5 * v[group == "treatment"] + 0.5 * v[group == "control"]))
```

```
# A tibble: 1 x 1
      SMD
    <dbl>
1 -0.0904
```

```r
# Permutation test: is this imbalance beyond what chance assignment produces?
obs_bal <- expt |>
  specify(response = body_mass_g, explanatory = group) |>
  calculate(stat = "diff in means", order = c("treatment", "control"))

null_bal <- expt |>
  specify(response = body_mass_g, explanatory = group) |>
  hypothesize(null = "independence") |>
  generate(reps = 500, type = "permute") |>
  calculate(stat = "diff in means", order = c("treatment", "control"))

get_p_value(null_bal, obs_bal, direction = "both")
```

```
# A tibble: 1 x 1
  p_value
    <dbl>
1   0.408
```

(In lecture the Love plot was made with the `cobalt` package: `bal.tab(group ~ x1 + x2 + x3, data = df, s.d.denom = "pooled") |> plot()`.)

6 Prediction: Least Squares and R^2

6.1 Least squares recap

The linear model $\hat{y} = b_0 + b_1 x_1 + \cdots + b_p x_p$ is fit by the **method of least squares**: choose the coefficients minimizing the residual sum of squares. In R, `lm(y ~ x1 + x2, data = ...)`; predict on new data with `predict(m, newdata = ...)`; and compare a prediction "by hand" by plugging the new x values into the fitted equation (see Review 1 for interpreting coefficients and dummy variables).

6.2 RSS, TSS, SSR, and R^2

For each observation there are three values: the data y_i, the fit $\hat{y}_i$, and the overall mean $\bar{y}$. Three sums of squares measure the distances among them:

$$RSS = \sum_i (y_i - \hat{y}_i)^2 \qquad TSS = \sum_i (y_i - \bar{y})^2 \qquad SSR = \sum_i (\hat{y}_i - \bar{y})^2$$

- **RSS** (residual): data to fit — what the model leaves unexplained.
- **TSS** (total): data to its mean — total variability in y (n times the variance of y).
- **SSR** (regression; careful, *not* RSS): fit to the mean — variability the model explains.

For least squares fits these obey the "$\hat{y}$ sandwich" (a Pythagoras-style identity, from residuals being orthogonal to $\hat{y} - \bar{y}$):

$$RSS + SSR = TSS \qquad \Longrightarrow \qquad R^2 = 1 - \frac{RSS}{TSS} = \frac{SSR}{TSS}$$

R^2 is the proportion of the variability in y explained by the model: always in $[0, 1]$; 0 = no better than predicting $\bar{y}$ for everyone; 1 = perfect fit (all residuals 0).

```
m1 <- lm(body_mass_g ~ flipper_length_mm, data = penguins_c)

# From glance() ...
glance(m1) |> select(r.squared)
```

```
# A tibble: 1 x 1
  r.squared
      <dbl>
1     0.762
```

```
# ... and by hand from the three sums of squares
penguins_c |>
  mutate(y_hat = fitted(m1)) |>
  summarize(RSS = sum((body_mass_g - y_hat)^2),
            TSS = sum((body_mass_g - mean(body_mass_g))^2),
            R2  = 1 - RSS / TSS)
```

```
# A tibble: 1 x 3
        RSS        TSS    R2
      <dbl>      <dbl> <dbl>
1 51211963. 215259666. 0.762
```

6.3 Why R^2 alone can mislead

Adding a predictor to a least-squares model can **never lower** the training R^2 — it always stays the same or goes up, even if the new predictor is noise. (Same for raising a polynomial degree.) So training R^2 *cannot detect overfitting*: chasing the highest R^2 on the data you fit with will reward models that memorize your particular data set rather than learn the general trend.

```r
m2 <- lm(body_mass_g ~ flipper_length_mm + bill_length_mm, data = penguins_c)
m3 <- lm(body_mass_g ~ flipper_length_mm + bill_length_mm + bill_depth_mm,
         data = penguins_c)

c(m1 = glance(m1)$r.squared,
  m2 = glance(m2)$r.squared,
  m3 = glance(m3)$r.squared)   # monotonically non-decreasing
```

```
       m1        m2        m3
0.7620922 0.7627425 0.7639367
```

7 Overfitting and the Train / Test Split

Overfitting: a model tailored so closely to the specific data it was fit on (its noise included) that it predicts *new* data poorly. Classic symptom: a high-degree polynomial that wiggles through every training point.

The fix (workflow): never fit and evaluate on the same data.

1. **Split**: randomly partition the rows, e.g. 80% **training set** / 20% **testing set** (disjoint!).
2. **Fit** the candidate models on the training set only.
3. **Evaluate** each model on the testing set (R^2, MSE, or MCR for classification). A model that scores well on training but much worse on testing is overfit; choose the model with the best *testing* performance.

```r
set.seed(20)
# Step 1: split
penguins_split <- penguins_c |>
  mutate(set_type = sample(c("train", "test"), size = n(),
                           replace = TRUE, prob = c(0.8, 0.2)))
train <- penguins_split |> filter(set_type == "train")
test  <- penguins_split |> filter(set_type == "test")

# Step 2: fit on training data only
lm_slr  <- lm(body_mass_g ~ flipper_length_mm, data = train)
lm_poly <- lm(body_mass_g ~ poly(flipper_length_mm, degree = 10, raw = TRUE),
              data = train)

# Step 3: evaluate both on the testing set (R^2 computed by hand)
test |>
  mutate(pred_slr  = predict(lm_slr,  newdata = test),
```

```
        pred_poly = predict(lm_poly, newdata = test)) |>
  summarize(TSS      = sum((body_mass_g - mean(body_mass_g))^2),
            R2_slr   = 1 - sum((body_mass_g - pred_slr)^2)  / TSS,
            R2_poly  = 1 - sum((body_mass_g - pred_poly)^2) / TSS)
```

```
# A tibble: 1 x 3
        TSS R2_slr R2_poly
      <dbl>  <dbl>   <dbl>
1 60040108.  0.821   0.850
```

Training R^2 will always favor the 10-degree polynomial; testing R^2 tells you whether that complexity actually helped. A gap between training and testing performance is the signature of overfitting.

8 Logistic Regression

8.1 The model

In **classification** the response Y is categorical — here 0/1 — and we predict the *probability* $P(Y = 1 \mid X)$. A straight line $\hat{p} = b_0 + b_1 x$ won't do (it escapes $[0, 1]$), so we keep the linear score

$$\hat{z} = b_0 + b_1 x_1 + \cdots + b_p x_p$$

and push it through the **standard logistic (sigmoid) function**

$$s(z) = \frac{1}{1 + e^{-z}} \qquad \Longrightarrow \qquad \hat{p} = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + \cdots + b_p x_p)}}.$$

$s(z)$ maps any real number into $(0, 1)$: large $z \to 1$, very negative $z \to 0$, and $s(0) = 0.5$. Because a linear model sits inside a fixed transformation, this is a *generalized linear model*. To **classify**, apply a threshold: $\hat{y}_i = 1$ if $\hat{p}_i > 0.5$ (say), else 0. The sign of a coefficient still tells direction: $b_1 > 0$ means larger $x_1 \to$ higher predicted probability of 1.

8.2 Computing a probability by hand

Suppose the fitted model for spam is $\hat{z} = b_0 + b_1 \cdot$ `log_num_char` with $b_0 = -1.72$ and $b_1 = -0.54$, and a new email has `log_num_char` $= 2$. Then

$$\hat{z} = -1.72 - 0.54 \times 2 = -2.8, \qquad \hat{p} = \frac{1}{1 + e^{2.8}} \approx 0.057,$$

so classify as *not spam* ($0.057 < 0.5$). Two steps, always: linear score first, then logistic.

```
1 / (1 + exp(-(-1.72 - 0.54 * 2)))
```

```
[1] 0.05732418
```

8.3 Fitting with `glm()` and evaluating with MCR

`glm(..., family = "binomial")` is the logistic analog of `lm()`. With `predict(..., type = "response")` you get $\hat{p}_i$; threshold with `ifelse()`; then evaluate with the **misclassification rate**:

$$\text{MCR} = \frac{\#\text{FP} + \#\text{FN}}{\#\text{ predictions}}$$

where a **false positive** predicts 1 for a true 0 and a **false negative** predicts 0 for a true 1. (These mirror Type I / Type II errors from testing.)

```
# Predict a penguin's sex from body mass.
# glm() treats the SECOND factor level as "1"; here levels are female, male,
# so the model predicts P(male).
m_log <- glm(sex ~ body_mass_g, data = penguins_c, family = "binomial")
tidy(m_log) |> select(term, estimate, std.error)
```

```
# A tibble: 2 x 3
  term        estimate std.error
  <chr>          <dbl>     <dbl>
1 (Intercept) -5.16     0.724
2 body_mass_g  0.00124  0.000173
```

```r
penguins_c |>
  mutate(p_hat = predict(m_log, newdata = penguins_c, type = "response"),
         y_hat = ifelse(p_hat > 0.5, "male", "female")) |>
  summarize(MCR = mean(y_hat != sex))
```

```
# A tibble: 1 x 1
    MCR
  <dbl>
1 0.390
```

```r
# Where do the errors come from? Cross actual vs predicted (4 cells):
penguins_c |>
  mutate(p_hat = predict(m_log, type = "response"),
         y_hat = ifelse(p_hat > 0.5, "male", "female")) |>
  count(sex, y_hat)
```

```
# A tibble: 4 x 3
  sex    y_hat      n
  <fct>  <chr>  <int>
1 female female   109
2 female male      56
3 male   female    74
4 male   male      94
```

Exam move

Same four-step recipe as regression, with two substitutions: model form is logistic-of-linear instead of linear, and the metric is MCR instead of R^2/MSE. Everything else carries over — fit on the training set, evaluate MCR on the testing set, and inspect false positives vs false negatives separately (a low overall MCR can hide a model that never catches the rare class).
